
sentiment analysis service

sổ tay làm chủ dự án

ưu tiên triển khai, bám mã nguồn, tối ưu cho onboarding

mô hình phân công trách nhiệm

vai trò	trách nhiệm chính
tech lead	quản trị kiến trúc, lộ trình kỹ thuật, rà soát bảo mật, sẵn sàng production
ml engineering lead	pipeline huấn luyện, chiến lược dữ liệu, đánh giá, lo-RA/PEFT, MLflow, DVC
ai engineering lead	runtime suy luận, ONNX, SHAP, ABSA, nhận diện ngôn ngữ, pipeline dự đoán
solution engineering lead	thiết kế end-to-end, luồng nghiệp vụ, tích hợp, giám sát, triển khai
backend engineering lead	FastAPI, hợp đồng api, validation, batch, api đánh giá
ui/ux engineering lead	Angular frontend, ui giải thích, ui upload batch, trực quan hoá

thứ tự nguồn sự thật

source code → cấu hình runtime → tests → report hiện có → tài liệu cuối cùng

Contents

1	tóm tắt điều hành	3
2	bối cảnh kinh doanh	3
2.1	ai sử dụng	3
2.2	thành công được đo như thế nào	4
2.3	mismatch quan trọng	4
3	mô hình tinh thần của toàn hệ thống	4
3.1	một trang nhìn tổng quát	4
3.2	góc nhìn của engineer	5
4	kiến trúc sâu	5
4.1	kiến trúc runtime	5
4.2	tradeoff thiết kế	6
5	luồng thực thi runtime	6
5.1	startup ứng dụng	6
5.2	luồng prediction	6
5.3	các luồng khác	6
6	frontend	7
6.1	cấu trúc component	7
6.2	state và local storage	7
6.3	rủi ro contract frontend/backend	7
7	backend	7
7.1	api surface	8
7.2	xử lý lỗi	8
8	ai inference	8
8.1	model layer làm gì	8
8.2	khái niệm cốt lõi	8
8.3	ý nghĩa khi vào production	9
9	ml training	9
9.1	pipeline huấn luyện	9
9.2	balancing và loss	9
10	data lineage	10

11 dvc pipeline	10
12 onnx pipeline	11
13 api reference	11
14 security review	11
15 observability	12
16 performance analysis	12
17 failure analysis	12
18 technical debt register	12
19 ownership matrix	13
20 raci matrix	13
21 team responsibilities	13
22 debugging handbook	13
22.1 backend không khởi động	14
22.2 prediction lỗi	14
22.3 batch upload lỗi	14
23 incident response guide	14
24 new engineer onboarding guide	14
25 production readiness assessment	14
26 roadmap	15

1 tóm tắt điều hành

repository này là một sản phẩm sentiment analysis dạng container hoá, kèm theo toàn bộ hệ thống ml offline tạo ra artifact cho runtime. dòng online là Angular → Nginx → FastAPI → model inference. dòng offline là DVC → preprocessing → finetuning → evaluation → ONNX export. hệ thống còn có monitoring, experiment tracking và explainability.

được án tồn tại vì feedback dạng văn bản có giá trị lớn nhưng đọc tay thì chậm và tốn công. codebase biến text thô thành output có cấu trúc: sentiment tổng thể, sentiment theo aspect, cò sarcasm và giải thích ở mức token. điều này hữu ích cho phân tích sản phẩm, triage support và demo explainable ai.

lý do một kỹ sư giàu kinh nghiệm sẽ thiết kế theo hướng này:

- tách phần phục vụ inference khỏi phần tạo artifact offline để runtime gọn
- giữ backend thành một process duy nhất để vận hành đơn giản
- dùng PEFT/LoRA để giảm chi phí huấn luyện và kích thước artifact
- dùng ONNX để có runtime dự đoán ổn định hơn
- dùng DVC và MLflow để tái lập được dữ liệu, huấn luyện và đánh giá

2 bối cảnh kinh doanh

2.1 ai sử dụng

- người dùng cuối muốn có câu trả lời sentiment nhanh
- analyst muốn xem batch CSV
- ml engineer muốn huấn luyện và export adapter
- backend engineer muốn duy trì hành vi api
- platform engineer muốn vận hành container và metrics

2.2 thành công được đo như thế nào

Table 1: tín hiệu thành công

khu vực	tín hiệu	ý nghĩa
product	sentiment và absa chính xác	tạo giá trị thật cho người dùng
runtime	latency thấp trên <code>/predict</code> và <code>/batch_predict</code>	trải nghiệm tương tác mượt
reliability	health endpoint và xử lý lỗi an toàn	có thể monitor khi deploy
ml	các run huấn luyện/đánh giá tái lập được	mô hình có thể tạo lại an toàn
operations	Prometheus, Grafana, MLflow, logs	operator có thể debug và quan sát

2.3 mismatch quan trọng

frontend có language selector, nhưng `AppComponent` hiện tại bỏ mất ngôn ngữ đã chọn trước khi gửi request. vì vậy ui trông giống như hỗ trợ chọn ngôn ngữ tương minh, trong khi backend thực tế tương ứng dựa vào detection. đây là mismatch ở implementation, không chỉ là lệch tài liệu.

3 mô hình tinh thần của toàn hệ thống

3.1 một trang nhìn tổng quát

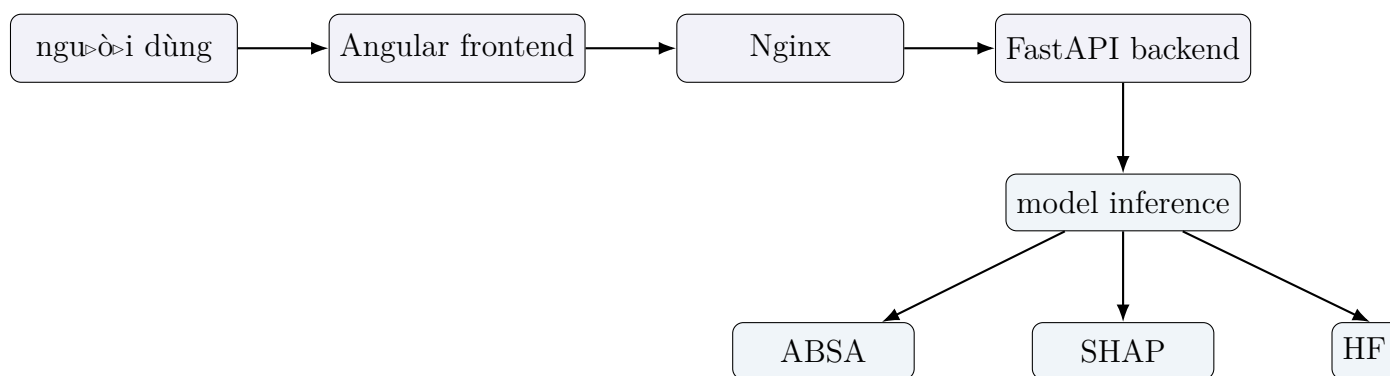


Figure 1: mô hình runtime ở mức cao

ở runtime, backend là trung tâm. nó load model trong lúc startup, warm up pipeline ABSA, mở health và metrics endpoint, rồi trả lời mọi request dự đoán. các thành phần còn lại tồn tại để hệ thống dễ dùng, dễ quan sát và có thể tái lập.

3.2 góc nhìn của engineer

- **frontend:** compose message, lịch sử chat, explain panel, modal upload batch
- **backend:** route FastAPI, validation, error mapping, metrics middleware
- **model layer:** baseline HF, finetuned PEFT, ONNX runtime
- **offline ml:** download data, preprocess, validate, finetune, export
- **operations:** Prometheus, Grafana, MLflow, Docker Compose, Nginx

4 kiến trúc sâu

4.1 kiến trúc runtime

Table 2: thành phần chính và trách nhiệm

thành phần	file chính	trách nhiệm
frontend	app/sentiment-analysis-chatbot/src/app/*	tuong tác nguoi dùng, state, api calls, ui explain
backend	src/main.py	validation, routing, startup, error handling, metrics
model	src/model/baseline.py	sentiment, sarcasm, ABSA, SHAP, batch prediction
ONNX	src/model/onnx_inference.py	load session và tối ưu suy luận
training	src/scripts/run_finetuning.py, src/training/*	huấn luyện adapter, class weighting, MLflow tags
data	src/data/*	download, preprocess, validate, split, report
monitoring	src/monitoring/*, infra/*	metrics, dashboard, alert
deployment	docker-compose.yml, Dockerfile	orchestration và đóng gói

4.2 tradeoff thiết kế

- một backend service dễ vận hành hơn microservice mesh
- một abstraction cho model giúp api layer không bị lộ chi tiết huấn luyện/runtime
- ONNX là tối ưu hoá, không phải nguồn sự thật; code huấn luyện vẫn ở PyTorch/Hugging Face
- ABSA và SHAP có giá trị, nhưng có chi phí; code để explicit để caller chủ động trả giá

5 luồng thực thi runtime

5.1 startup ứng dụng

- `src/main.py` tạo FastAPI app và gắn lifespan handler
- lifespan đọc `MODEL_MODE`
- tạo `ModelConfig`
- khởi tạo `BaselineModelInference`
- gọi `preload()` để ABSA sẵn sàng trước khi có traffic

5.2 luồng prediction

1. người dùng nhập text ở `ChatInputComponent`
2. frontend service post text lên `/predict`
3. `src/main.py` validate request bằng `PredictRequest`
4. backend resolve language
5. `BaselineModelInference.predict_single()` chạy model
6. response được ghép thành `PredictResponse`
7. ui render sentiment, confidence, aspects và sarcasm

5.3 các luồng khác

- **health check:** chỉ xác nhận model đã load, không phải toàn bộ platform
- **explain:** tìm user message trước đó rồi gọi `/explain`

- **batch:** parse CSV, validate cột text, dự đoán từng dòng
- **evaluation:** background task và CLI cùng dùng chung abstraction model

6 frontend ---

6.1 cấu trúc component

- AppComponent: shell, health polling, chat history, modal state
- SentimentAnalysisService: http client và message state
- ChatInputComponent: text input, chọn ngôn ngữ, voice input
- MessageBubbleComponent: render message và SHAP
- BatchUploadComponent: upload CSV và export kết quả

6.2 state và local storage

- chat history lưu trong local storage
- dark mode cũng lưu trong local storage
- service giữ danh sách message đang active làm nguồn render chính

6.3 rủi ro contract frontend/backend

- language selector chưa đi hết đường end-to-end
- validation batch phía client chỉ là kiểm tra bề mặt
- ui giả định response schema ổn định; đổi field hoặc label phải đồng bộ

7 backend ---

7.1 api surface

Table 3: endpoint đã triển khai

method	path	mục đích
GET	/health	trạng thái readiness và model
POST	/predict	dự đoán single text
POST	/explain	giải thích SHAP
POST	/batch_predict	batch prediction từ CSV
GET	/batch_status/{job_id}	endpoint trạng thái mock
POST	/evaluate	trigger đánh giá background
GET	/evaluate/status	trạng thái evaluation trong memory
GET	/metrics	endpoint scrape cho Prometheus

7.2 xử lý lỗi

- validation dùng Pydantic schema
- model errors được đổi thành JSON response hoặc lỗi 4xx/5xx
- /batch_status là mock, không backing bằng job queue thật

8 ai inference

8.1 model layer làm gì

model layer che nhiều chế độ inference sau cùng một interface. đây là quyết định kiến trúc quan trọng. api layer không cần biết runtime đang dùng baseline HF checkpoint, PEFT adapter hay ONNX session.

8.2 khái niệm cốt lõi

- **tokenizer:** biến text thành token, id, attention mask và tensor
- **logits:** điểm thô từ classifier head trước softmax
- **softmax:** biến logits thành xác suất có tổng bằng 1
- **confidence:** xác suất của nhãn được chọn
- **LoRA / PEFT:** adapter nhẹ gắn vào backbone
- **ABSA:** rút sentiment theo aspect như food hoặc service

- **SHAP**: giải thích mức token cho quyết định của model
- **sarcasm**: cò riêng cho hành vi kiểu mỉa mai/irony

8.3 ý nghĩa khi vào production

- ONNX làm runtime dự đoán ổn định hơn
- ABSA và SHAP nên là chi phí opt-in
- batch prediction bỏ ABSA vì chạy ABSA theo từng dòng sẽ quá đắt
- adapter finetune phải được version hoá cùng data và code

9 ml training

9.1 pipeline huấn luyện

- dữ liệu được tải theo từng task
- file raw được chuẩn hoá và split
- validation ép đúng shape và chất lượng label
- finetuning train sentiment và sarcasm adapter bằng PEFT/LoRA
- evaluation log metric sang MLflow
- export merge adapter và sinh artifact ONNX

9.2 balancing và loss

- class imbalance được xử lý bằng weighted loss
- oversampling được dùng nó cải thiện đại diện lớp thiểu số
- smoke mode chỉ để verify nhanh, không dùng làm benchmark chất lượng

10 data lineage

Table 4: artifact chính

artifact	producer	consumer	vòng đời
data/raw/*	downloader scripts	preprocessing / training	kéo tù dataset ngoài
data/processed/*	preprocess pipeline	baseline evaluation / training	artifact sinh ra, ver- sion hoá bởi DVC
models/adapters/*	finetuning scripts	ONNX export / runtime	output của train- ing
models/onnx/*	ONNX exporter	runtime inference	artifact de- ploy được
data/reports/*	validation / evaluation scripts	con người / MLflow	artifact au- dit và re- view

11 dvc pipeline

- download: lấy raw corpus
- preprocess: clean, normalize, split
- validate: enforce quality gates

- `evaluate_baseline`: chấm điểm baseline
- `download_sarcasm` và `download_sentiment`: chuẩn bị data theo task
- `finetune_sarcasm` và `finetune_sentiment`: train adapter
- `evaluate_finetuned`: so sánh kết quả finetuned
- `export_onnx_sentiment` và `export_onnx_sarcasm`: đóng gói artifact runtime
- `benchmark_onnx`: đo hành vi runtime

12 onnx pipeline

ONNX tồn tại để giảm friction lúc runtime. exporter merge PEFT adapter vào base model, xuất static graph, rồi có thể tạo bản INT8. runtime loader chọn provider theo hardware. cách này predictable hơn nhiều so với việc ship cả stack finetune vào production.

13 api reference

Table 5: chi tiết api

method	path	request / response	ghi chú
GET	/health	không body / health schema	readiness only
POST	/predict	PredictRequest / PredictResponse	main user path
POST	/explain	ExplainRequest / ExplainResponse	path tốn chi phí, gọi khi cần
POST	/batch_predict	multipart file / batch schema	xử lý batch đồng bộ
GET	/batch_status/{job_id}	path param / mocked status	không phải queue thật
POST	/evaluate	không body / status JSON	trigger background task

14 security review

- không có authentication hoặc authorization

- CORS khá mở
- batch upload nhận file CSV do người dùng cung cấp
- build-time secrets có thể bị lộ nếu truyền sai cách
- Grafana mặc định mật khẩu admin là không an toàn cho production

15 observability

- backend expose metrics cho Prometheus
- Grafana hiển thị request rate, error rate và latency
- alert rules bao phủ high error rate và high inference latency
- tracing chưa được implement trong repo

16 performance analysis

- startup cost chủ yếu nằm ở model loading và ABSA warm up
- SHAP và ABSA là hai feature đắt nhất theo request
- batch throughput bị giới hạn vì inference vẫn chạy theo từng dòng
- memory usage tăng theo kích thước backbone và adapter state

17 failure analysis

- **startup failure:** thiếu model path hoặc artifact
- **prediction 500:** tokenizer/runtime/model không khớp
- **explain failure:** không tìm thấy user message hoặc SHAP lỗi
- **batch failure:** CSV lỗi, thiếu cột text, hoặc vượt upload limit
- **metrics missing:** lỗi middleware hoặc scrape config

18 technical debt register

- endpoint batch status là mock
- language selector phía frontend chưa đi hết đường

- docs và implementation lệch ở một vài chỗ
- chưa có auth hoặc rate limiting
- coverage test phía frontend còn yếu

19 ownership matrix

Table 6: tóm tắt ownership

lead	phạm vi sở hữu chính
tech lead	kiến trúc, roadmap, bảo mật, sẵn sàng production
ml engineering lead	huấn luyện, dữ liệu, đánh giá, DVC, MLflow
ai engineering lead	inference, ONNX, SHAP, ABSA, language detection
solution engineering lead	solution end-to-end, monitoring, deployment
backend engineering lead	FastAPI, validation, batch và evaluation API
ui/ux engineering lead	frontend UX, explainability, batch upload, visualization

20 raci matrix

- tech lead chịu trách nhiệm cuối cùng cho production readiness
- ml lead chịu trách nhiệm chất lượng training và evaluation
- ai lead chịu trách nhiệm correctness của inference và runtime performance
- backend lead chịu trách nhiệm api contract và request flow
- ui/ux lead chịu trách nhiệm user flow và độ rõ của giao diện
- solution lead chịu trách nhiệm tích hợp end-to-end

21 team responsibilities

project này dễ maintain nhất khi mỗi chức năng có chủ sở hữu rõ. các thay đổi cắt ngang, như thêm label mới, ngôn ngữ mới, hoặc model mode mới, phải được review cùng lúc bởi backend, ai, ml và ui vì nó đụng đến contract, dữ liệu huấn luyện, code inference và frontend.

22 debugging handbook

22.1 backend không khởi động

check `MODEL_MODE`, đường dẫn artifact, file DVC đã pull, environment variables và startup logs.

22.2 prediction lỗi

check request validation, ngôn ngữ hỗ trợ, tokenizer availability, model mode và ONNX session.

22.3 batch upload lỗi

check shape CSV, cột `text`, file size và Nginx upload limits.

23 incident response guide

1. xác định lỗi thuộc startup, request hay data
2. xem backend logs trước vì backend nắm model loading và request handling
3. kiểm tra health và metrics endpoint
4. đối chiếu version artifact runtime với lần training/export thành công gần nhất
5. rollback artifact trước khi đổi code nếu lỗi chỉ nằm ở runtime

24 new engineer onboarding guide

- đọc `src/main.py`, `contracts/schemas.py` và `src/model/baseline.py` trước
- sau đó đọc frontend service và batch component
- tiếp theo đọc training và ONNX export scripts
- cuối cùng xem DVC, Docker và monitoring configs

25 production readiness assessment

repository này đã gần với một ứng dụng ml production-style, nhưng vẫn còn gap: không có auth, không có rate limiting, batch status endpoint là mock, và một vài contract mismatch vẫn tồn tại. kiến trúc đi đúng hướng, nhưng vẫn cần hardening trước production.

26 roadmap

Table 7: lộ trình đề xuất

mốc	trọng tâm
3 tháng	đóng các gap contract, bảo vệ api, sửa semantics batch, tăng test
6 tháng	cải thiện observability, thêm traceability, tinh chỉnh ONNX và evaluation workflow
12 tháng	scale runtime topology, formalize artifact provenance và siết ownership boundary